SENIOR .NET INTERVIEW SERIES

C# & Language

OOP, records, pattern matching, delegates, reflection — the language questions every senior interview starts with.

20 Real Interview Questions, Answered

Swipe through — save this for your next interview prep.

Q. What's the difference between a class, a record, and a struct — and when do you pick each?

A. Classes are reference types with identity-based equality — your default for entities and services. Records are reference types with value-based equality and built-in immutability support — ideal for DTOs and domain values. Structs are value types stored inline — pick them for small, short-lived data (under ~16 bytes) to avoid heap allocations.

```
public record Money(decimal Amount, string Currency);
var a = new Money(10m, "USD");
var b = a with { Amount = 20m }; // non-destructive
```

Q. Explain boxing and unboxing. Why do seniors care?

A. Boxing wraps a value type in a heap-allocated object; unboxing extracts it back. Each box is a hidden allocation plus GC pressure. It sneaks in via non-generic collections, string.Format with value types, and casting structs to interfaces. Generics exist largely to eliminate it — List<int> never boxes, ArrayList always does.

Q. IEnumerable<T> vs IQueryable<T> — what actually changes?

A. IEnumerable executes in memory with LINQ-to-Objects delegates. IQueryable builds an expression tree that a provider (like EF Core) translates to SQL, so filtering happens in the database. Calling .AsEnumerable() too early pulls the whole table into memory and filters there — a classic production performance bug.

Q. What is deferred execution in LINQ, and where does it bite you?

A. A LINQ query is a description, not a result — it runs only when enumerated. It bites you when: you enumerate twice and execute the query twice; you capture a loop variable that changes before enumeration; or a DbContext is disposed before the query runs, throwing ObjectDisposedException. Materialize with ToList() when you need a stable snapshot.

Q. Explain delegates, Func/Action, and events — how do they relate?

A. A delegate is a type-safe reference to a method. Func<T> and Action are generic delegate types so you rarely declare custom ones. An event is a delegate field with restricted access: outside subscribers can only += / -=, only the declaring class can invoke it. Events are the publisher/subscriber primitive; delegates are the underlying mechanism.

Q. What are the latest pattern matching features you actually use?

A. Property patterns, relational and logical patterns, and list patterns turn multi-branch if-chains into declarative switch expressions that the compiler checks for exhaustiveness.

```
var fee = order switch
{
    { Total: > 1000 }        => 0m,
    { Items.Count: > 5 }     => 5m,
    { Customer.IsVip: true } => 2m,
    _                        => 10m
};
```

Q. How does exception handling affect performance, and what's your policy?

A. Try blocks are nearly free; throwing is expensive (stack unwind + trace capture). So: never use exceptions for control flow, use TryParse-style APIs on hot paths, catch only what you can handle, and rethrow with 'throw;' not 'throw ex;' — the latter destroys the original stack trace. Use exception filters (catch...when) to avoid catch-and-rethrow.

Q. When would you reach for reflection, and what are the alternatives?

A. Reflection powers serializers, DI containers, and plugin discovery — but it's slow and breaks with trimming/AOT. Alternatives: cache MemberInfo lookups, compile expression trees once, or better, use source generators (like System.Text.Json does) to move the work to compile time with zero runtime cost.

Q. What does 'dynamic' do under the hood, and when is it acceptable?

A. dynamic defers member binding to runtime via the DLR, with call-site caching. You lose compile-time safety and IntelliSense, and every call has overhead. Acceptable for genuine dynamic scenarios: COM interop, parsing unknown JSON shapes, or scripting bridges. In domain code it's almost always a design smell.

Q. How do you design serialization for a public API — System.Text.Json specifics?

A. Prefer System.Text.Json: it's allocation-friendly and source-generator ready. Key senior points: use JsonSerializerContext for AOT/trimming, handle polymorphism with [JsonPolymorphic], never serialize domain entities directly (use DTOs), version contracts additively, and treat unknown fields as ignorable so old clients don't break.

Q. A junior's PR compares two strings with == and it fails randomly in production. What do you look for?

A. Likely culture or casing: they're comparing user input or parsed data with default ordinal semantics somewhere and culture-sensitive ToLower/ToUpper elsewhere. Standardize on string.Equals with an explicit StringComparison (OrdinalIgnoreCase for identifiers, culture-aware only for display). Also check for hidden whitespace/normalization issues in the inputs — Trim and Unicode normalization.

Q. You see 'async void', 'public fields', and 'GC.Collect()' in a code review. Rank the danger and explain.

A. async void first: exceptions bypass every handler and can kill the process — must fix unless it's a UI event handler. GC.Collect() second: it usually masks a leak while wrecking GC pacing — find why memory grows instead. Public fields last: bad encapsulation and binding/serializer quirks, but rarely a production incident. Prioritizing by blast radius is the senior part.

Q. When has 'var' hurt you, and what's your team convention?

A. var hides unintended types at boundaries: var result = service.Get() compiling to a Task you forgot to await, or an IQueryable you thought was a list. Convention: var when the type is obvious from the right side (new expressions, casts, LINQ chains); explicit types for method returns and primitives in tricky code. Consistency beats dogma — put it in .editorconfig.

Q. Your DTO records break after an EF Core upgrade because of constructors. What's the deeper lesson?

A. Records with positional parameters rely on constructor binding; ORMs, serializers, and mappers each have their own construction rules — parameterless constructor expectations, init-only setters, required members. Lesson: records are perfect at the edges (API contracts, messages) but be deliberate using them as persisted entities — EF's change tracking and value-based equality interact badly with entity identity.

Q. Explain covariance and contravariance with a bug you've seen.

A. Covariance (out T): IEnumerable<Dog> is an IEnumerable<Animal> — safe because T only flows out. Contravariance (in T): IComparer<Animal> works as IComparer<Dog>. The classic bug is array covariance: passing Dog[] as Animal[] compiles, then storing a Cat throws ArrayTypeMismatchException at runtime — a reason to prefer generic collections, which enforce this at compile time.

Q. A teammate wants to add an extension method on 'object'. Talk them off the ledge.

A. It pollutes IntelliSense on every type in every consuming file, invites misuse far from its intent, and creates resolution ambiguity as more extensions pile up. Extensions shine on specific types or interfaces you can't modify. If the logic really is universal, a static helper with an honest name is clearer than fake instance syntax on everything.

Q. What actually changes when you mark a struct 'readonly', and when do you bother?

A. The compiler enforces immutability and — the real win — stops making defensive copies when the struct is accessed through 'in' parameters or readonly fields. Non-readonly structs in those positions get silently copied on every member call, which quietly erases the performance you used a struct for. Bother whenever the struct is meant to be immutable, which should be almost always.

Q. You need to add behavior to a sealed third-party class. What are your options, honestly ranked?

A. Composition first: wrap it behind your own interface — also your seam for testing. Extension methods for pure convenience helpers, no state. Adapter if its shape doesn't fit your domain. Reflection into privates is last resort — it breaks on their next release. The anti-answer interviewers listen for: complaining that it should be unsealed instead of designing around it.

Q. What are nullable reference types actually doing, and why do teams half-adopt them badly?

A. Compile-time flow analysis over annotations — zero runtime behavior change; string vs string? is documentation the compiler checks. Teams fail by enabling them with 8,000 warnings, suppressing with ! everywhere, and trusting annotations at unchecked boundaries (JSON, DB, reflection) where nulls still arrive. Adopt per-project, treat warnings as errors, validate at the edges.

Q. How do you decide between throwing, returning null, and a Result type?

A. Throw for exceptional, unexpected failures — broken invariants, unavailable dependencies. Return null (with NRT annotations) only for genuine 'not found' lookups where callers expect absence. Result/OneOf types for expected domain outcomes — validation failures, business rule rejections — forcing callers to handle both paths. Pick per API and be consistent; a codebase mixing all three styles randomly is the real failure.

That's a wrap.

20 C# & Language questions that actually come up in senior .NET interviews — with the answers experienced engineers give.

NEXT IN THE SERIES

Async, Threading & Memory

Found this useful?

Follow for the rest of the series,
and save this post for your interview prep.